Advanced Pygame Zero Cheat Sheet

CS4040 · Unit 5

Assets

Folder	Holds	File types
`images`	pictures for Actors	`.png`
`sounds`	sound effects	`.wav` or `.ogg`
`music`	background music	`.ogg` or `.mp3`

Asset names follow the same rules as Python variables: all lowercase, no spaces, only letters, numbers and underscores, and they must begin with a letter. Refer to an asset by its name without the extension: `"alien"`, not `"alien.png"`.

This module adds two more folders: keep your spritesheets in `sprites`, and your tilemaps, tilesets, and tilesheets in `maps`.

Actor extensions

Additional functionality added to Actors:

```
player.angle  # gets current angle in degrees
player.angle += 1  # increments angle by 1
player.scale  # gets current Actor scaling factor
player.scale = 2.0  # sets the Actor scaling factor
player.flip_v = True  # flips the Actor vertically
player.flip_v = False  # turns off vertical flip, default
player.flip_h = True  # flips the Actor horizontally
player.flip_h = False  # turns off horizontal flip, default
```

```
player.opacity = 0.6  # sets opacity, from 0.0 to 1.0
player.visible = False  # hides it; still moves and collides
```

Collision rectangle

A custom collision rectangle is centered on the Actor. The first line makes it 10 wide and 20 tall; the second makes it 50 wide and 20 tall, with its center moved by (5, −5).

```
player.collision_rect = (10, 20)  # (width, height)
player.collision_rect = (50, 20, 5, -5)  # (width, height, offset_x, offset_y)
```

In `draw()`:

```
player.draw_collision_rect()  # shows the collision rect, useful for debugging
```

Sprites

Use `Sprite` to make an animation from a spritesheet in the `sprites` folder for an Actor.

- `image_name`: sprite sheet filename
- `frame_width`: width of each frame in pixels
- `frame_height`: height of frame in pixels
- `row` (optional): row to start from, counting from 0. Defaults to `0`.
- `frames` (optional): number of frames. Defaults to the whole row.
- `fps` (optional): frames per second. Defaults to `10`.
- `transparent_color` (optional): color to make see-through. Defaults to none.
- `loop` (optional): `False` plays once. Defaults to `True`.

```
# define Sprites
idle = Sprite("rabbit", frame_width=16, frame_height=16,
              row=0, frames=2, fps=3)
walk_left = Sprite("rabbit", frame_width=16, frame_height=16,
                   row=6, frames=2, fps=10)
walk_right = Sprite("rabbit", frame_width=16, frame_height=16,
                    row=7, frames=2, fps=10)

# make an Actor that plays a Sprite
rabbit = Actor(idle)
```

Controlling animations

`fox.sprite` gets the Sprite the Actor is playing. Setting it switches the animation. Setting the same Sprite again restarts it only if it has finished, so it is safe to set every frame.

```
idle = Sprite("fox", 32, 32, row=0)
attack = Sprite("fox", 32, 32, row=3, loop=False)  # plays once
fox = Actor(idle)
```

```
fox.sprite = attack  # same Sprite again: replays only if it has finished
if fox.animation_finished:  # True once a loop=False Sprite ends
    fox.sprite = idle
fox.frame  # current frame number, starting at 0
fox.pause_animation()  # freezes on the current frame
fox.resume_animation()  # resumes animation on current frame
```

Tile map constants

Helpful constants for 2D games using a tilemap.

```
# our tile map constants
TILE_SIZE = 18  # length of a tile, in pixels
MAP_WIDTH = 30  # width of the tilemap, in tiles
MAP_HEIGHT = 20  # height of the tilemap, in tiles
SCALE = 2  # global scaling factor
# can use to compute screen dimensions
WIDTH = TILE_SIZE * MAP_WIDTH * SCALE
HEIGHT = TILE_SIZE * MAP_HEIGHT * SCALE
```

Tilesheets, tilesets, and tilemaps

- **Tilesheet**: a `.png` image file that contains many small images (tiles) arranged in a grid. Each tile is a piece of the world (grass, wall, water, etc.). This is the art asset created by the artist.
- **Tileset**: a `.tsx` file created in Tiled that defines how to use a tilesheet. It tells Tiled things like tile size, spacing, and any extra data. It references a tilesheet. (*Tileset* and *tilesheet* are often used interchangeably.)
- **Tilemap**: a `.tmx` file created in Tiled that defines the actual level layout. It uses one or more tilesets and organizes tiles into layers (ground, walls, hazards, etc.). This is the level design.

Note: the tilesheet, tileset, and tilemap must all be stored in the `maps` folder of your project.

Importing a tilemap

Use `load_tile_map_actors()` to import your map made in Tiled into your game based on tile map layers.

- `tmx_name`: path to the TMX file defining the tilemap, created in Tiled.
- `scale`: scaling factor applied to the map for display in Pygame.

To get individual lists of Actors from each Tiled layer, use the layer name. Using different layers allows you to treat each list of Actors differently.

```
map_layers = load_tile_map_actors("L1.tmx", scale=SCALE)
ground = map_layers["ground"]  # ground / background scenery to draw
walls = map_layers["walls"]  # obstacles for player to collide with
hazards = map_layers["hazards"]  # "bad" objects for player collision
hearts = map_layers["hearts"]  # "good" objects or collectables
```

Wall collisions

Example of how to handle collisions with walls. Repeat the same steps for movement in each cardinal direction.

```
# left keyboard button pressed
if keyboard.LEFT:
    rabbit.x -= 2  # move to left
    rabbit.sprite = walk_left  # change to left sprite
    # check for collision
    collision_index = rabbit.collidelist(walls)
    # did collision occur after moving to left?
    if collision_index != -1:
        # if so, put character's left edge on right edge of wall
        rabbit.left = walls[collision_index].right
```